

LovellLight Desk Lamp and Phone Application

Senior Project 2016

Wilson Holiday

Table of Contents

Introduction.....	1
Project Background and Purposoe.....	1
Research.....	2
Design.....	5
Problems Encountered.....	8
Appendices.....	11



Introduction

The name of this Senior Project is the LovellLight system. It consists of the LovellLight itself (made up of 3-D printed parts, a gooseneck lamp neck with lampshade, and many electronics components that are inside of the lamp that keep it running), as well as a companion phone app called the LovellLight Companion (LLC). The system works by having power provided to the light source and Arduino in the lamp through ports on the side, and using the app to either search for the Bluetooth in the lamp for pairing in an unpaired list, or choose the lamp for pairing through a paired list. Once the user has pressed on the list item containing the information for the user, the user waits for the connection to go through, and is then brought to a screen that allows them to change the LovellLight's light setting to their likings. Due to the nature of the gooseneck lamp part and the unlimited colors that the user can create, the LovellLight's system is entirely based around flexibility and practicality.

Project Background and Purpose

The idea for this senior project came from the many experiences I've had as a student at CSU. Every year, roommates with conflicting schedules must make preparations and considerations for each other, the most common being the use of the dorm room at night time. The use of a desk lamp is a pretty common form of compromise, as it allows one roommate to get any reading or writing for schoolwork or personal time done while another roommate utilizes the darkness of the rest of the room for rest or

preference. Not only can a desk lamp be useful for seeing material in one's personal desk in a dark room, but it can also be useful for adding a personal touch to a room when the light can utilize more colors than the base "white" light. Thus, the LovellLight concept was born. Utilizing a special, Adafruit manufactured ring light and a Ionic-developed phone application, the LovellLight allows users to control the ring light to make their desk lamp light any color that they can conceive and customize their work space, bringing together the practical and the personal. The purpose of this project is to make the LovellLight as efficient as possible in solving two things: being a lamp that a user can use with flexible light settings for various situations, and being a electronic piece that a user can also use to personalize their workspace.

Research

Microcontroller

For the components of the lamp, the Arduino Uno (rev 3) was chosen because of the vast amount of data available for tutorials for easier learning, as well as the versatile nature of the Microcontroller, being able to incorporate it into a system to make anything from a Door lock to a desk lamp. Using the Arduino Uno, a breadboard, a RGB LED included in the Arduino kit, several jumper wires and a basic Arduino-compatible Bluetooth module (HC-06), a basic prototype set up was built. This set up utilized a Arduino script and a Bluetooth terminal (BlueTerm, found in the Google PlayStore) to feed information to the Arduino, which in turn changed the LED based on the color values indicated. For the

final hardware set up, all of the basic parts remained the same except for the light source, which changed from an Arduino-powered LED to 12 LED ring light with a separate power source.

The Arduino has two special libraries installed that allows it to communicate with the ring light and the Bluetooth module: the Adafruit_Neopixel library and the Software Serial library. The Software Serial library is an Arduino Library that allows serial communication on the digital pins of the Arduino. The purpose of this library is to duplicate the functionality of the 2 pins on the Arduino dedicated for serial connections (0 and 1) and allow their functions and methods to be used by several other pairs of pins. It does have its limitations, as only some pins can be used for the reading (the RX functionality can be used through Software Serial to any pins except those dedicated to power or ground, or pin 13). The Adafruit Neopixel library is used to control the Neopixel LED on the ring light used in this project. It is available for download from the Adafruit website and works with most mainstream Arduino boards, including the Uno board used for the LovellLight. To use it, it must be installed and included in the Arduino sketch and have a few pieces of information specified when declaring a Neopixel variable. This information is: the number of Neopixels that the Neopixel hardware used will be utilizing or has, the pin number on the microcontroller that will be used to send information to the Neopixel hardware, and a value indicating the type of Neopixels that are connected (NEO_RGB in this case). Once the Neopixel variable has been declared using these values, it must be initiated in the Setup portion of the Arduino code (`variable_name.begin`; `variable_name.show()`); The second line initializes all pixels to off). Anything desired for the Neopixel light changes must be written in a way specifically for each hardware type. For example, to change pixel 10 of the ring light in the LovellLight setup, the Arduino code must have “`variable_name.setPixelColor(10, red_value, green_value, blue_value)`”, with the three unspecified values being integers between 0 and 255.

Lamp Shell

After finalizing the hardware parts of the lamp system, a decision needed to be made on finding a sufficient lamp shell to hold in all of the lamp components and to protect them from outside interference. After discussing the benefits of 3-D printing, and comparing it to using existing lamp shells, 3-D printing was chosen as the method of providing the lamp shell. In order to make the design of the lamp shell, the 3-D printing software SketchUp was used to prototype the lamp. For the final design, the created SketchUp sketches were transferred into a program called PTC Creo, which was then uploaded to a Maker's tool Works 3-D printer for the printing process.

Application

When I started conceptualizing the app for this project, I thought first about the people who would use something like this, and what they would use it for. College students, people in need of a desk lamp, and people looking for something to add personality to their workspace. Keeping this in mind, I thought to create an app that would be available for use on both iOS and Android to widen the audience that could use it. After looking into the different development environments, Ionic was chosen for the application building because of it's ability to be versatile across the platforms using Cordova plugins and the wide variety of libraries that could be incorporated. The next step, after choosing my development environment, was to search for plugins that would allow me to connect from the Arduino (via Bluetooth module) to the phone app. The Cordova plugin that handles the most of the controls is called the Bluetooth Serial Plugin, created by PhoneGap. This plugin is utilized in the app so that the user has the ability to: connect to BT devices previously paired to the device, search for BT devices

that haven't been paired to the device yet, cut the BT on the device on in order to gain access to BT related services, and connect to Bluetooth devices in order to either write to them or read from them.

Design

Microcontroller

The Arduino sketch utilizes a special library called the Software Serial library to handle all of it's functions. The function of the Software Serial library in the code is to provide the Arduino a way to successfully control and power the Bluetooth module, as well as to provide the sketch a way to read from the LovellLight Companion's signals. The Software Serial variable, titled Bluetooth, utilizes if statements in a main Arduino loop that constantly looks for incoming signals and interprets the incoming signals and the data they come with. The data that the Arduino sketch receives is an array of red, green, and blue values sent from the app that updates the lamps own red, green and blue values and feeds the information to the ring light.

The Bluetooth module is the middle man in providing information between the phone application and the Arduino. The module chosen for this project is a KEDSUM Arduino Wireless Bluetooth Transceiver Module. This module uses the Software Serial variable declared in the above paragraph to connect to the Arduino using two pins, 10 and 11 for the TX and RX pins respectively. Once power and ground has been provided to the Bluetooth module (via 3.3v power pin and Ground pin on the

Arduino), the module will start to blink and continue to blink until either depowered or connected to. Once connected to, the blinking light will stop blinking and stay on.

Lamp Shell

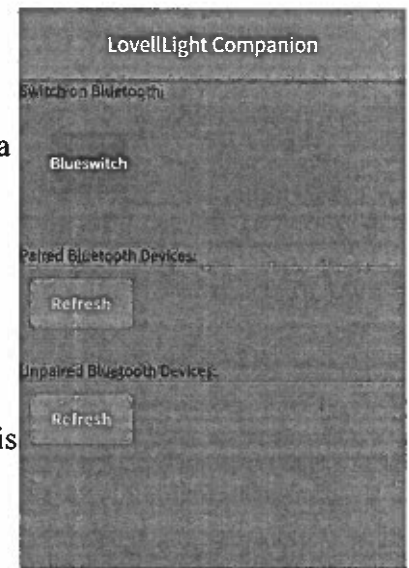
The Design of the Lamp shell is simple, but took incremental steps. The initial prototype was going to be a square, 4"x2"x1.75" black 3-D printed box that comes together from several parts: the box with the dimensions above but with groves in the corner, a top of approximately the same width and height but with solid material only on the upper 2/16" (and a 1/8" thick and 2" high "neck" on top of the top for the gooseneck) as the top for the box with legs that would fit in the groves of the base, in order to "lock" the model together. One of the shorter sides of the shell would have a small, rectangular opening that would allow the LovellLight user to provide power to its components via two ports.

The final design of the Lamp shell kept the shape of the lamp with a lot of the same dimension, but differing in the way that the top and one of the sides operated, a new piece added, as well as the way that the lamp "locked" together. The lamp base has seen the most change, filling in the groves on the side of the base, replacing the outer groves with inner grooves to place the top on, and making one of the smaller walls removable for quick fixes (and to assist in the lock in procedure via side grove). The top now has no legs, but is slightly narrower than the base in order to fit inside the groves provided. It also has it's own grove on the end that attaches to the top of the removable wall, keeping everything locked in place. The new addition to the lamp shell can't be seen from the inside, and exists as a table on the inside. The function this table is to keep the breadboard and the Arduino apart, allowing the Bluetooth module, the breadboard, the Arduino, and the ring light to have their own perspective areas. The table was made about as wide as the breadboard to allow some space between an inner wall and

the table for wires to run from under the table to the Arduino. The ring light is provided its own space by utilizing a 6" gooseneck that has the power and control wires run up through it and connect to the ring light. The ring light also has its own lampshade to assist in funneling light in one direction and adds a bit of a personal touch by being slightly transparent to add a "glow" effect.

Application

The LovellLight Companion app has an initial home screen consisting of a few designated areas:. The first of these areas is with a label that tells the user the status of the Bluetooth module on the phone and a button labeled "blueswitch" that allow the user to cut the Bluetooth on the phone on in order to utilize the app's other functions. The next areas directly below it is the Paired Device areas, consisting of a HTML list that allows the user to connect to devices by pressing the list item desired and a "Refresh" button that will update the list based on the list of paired devices in the phone's dedicated Bluetooth menu. The last area is the Unpaired Device zone, which has a basic layout similar to the Paired Device area except that the list is initially empty and must be update by clicking the "Refresh" button below the list. Waiting for a small period will cause the list to fill up with devices that the phone finds that aren't already included in the paired devices.



After the user has selected their LovellLight BT module from the list (HC-06), below the "Refresh" button the message "connecting" will show up, letting the user know to wait for the connection to take

place. Once it has, the user will be redirected to a different screen. This screen consists of just two designated areas. The first of these areas is designated for the RGB slides, where the user is allowed to move the slides back and forth. Once the slides are moved in the desired positions, pushing the “Send Color” button will use the information for all of the slides to update a three integer label at the bottom of slides detailing their color values and sends the appropriate values to the Arduino in the form an array with a string attached to the front and a newline at the end. Below the sliders and the RGB text field is a “Disconnect” button, that when pushed allows the user to disconnect from their device and brings them back to the first screen.

Problems Encountered

The issues that appeared when doing this system were few, but the few that did appear unfortunately resulted in cutting of additional desired parts and compromises. The smaller, easier to solve issues were related to the 3-D modeling in SketchUp and finding an appropriate Cordova plugin for the application. Initially, this Project was approved without an application and consisted of a system that utilized any random Bluetooth terminal from a phone’s appstore, but as app development became more reachable, that idea was scrapped. However, finding the appropriate plugin proved difficult, and the lack of prior knowledge of app development kept the app from being able to incorporate more ideas in a shorter time

span. Relating to the SketchUp modeling, the difficulty in solving that issue resulting in a combination of having to delay 3-D printing due to the student-available one being destroyed the week that the printing was scheduled to start, and finding out that SketchUp sketches aren't "watertight" (in the 3-D printing process, the printer would see holes that weren't initially visible to the user and would print them as such, resulting in a morphed form of the desired shape). These issues were solved with the incorporation of the PhoneGap plugin and using the PTC Creo program make the sketches more "watertight" and appropriate for the printing process. The PTC Creo program, as well as the alternative 3-D printer was supplied by Dr. Blanch, who also assisted in obtaining the gooseneck lamp part and creating the final lamp shell design.

The most significant issue encountered during the development of the LovellLight System was the attempted incorporation and eventual tossing of the idea of alternative "modes" for the LovellLight. Initially, along with the sliders and the disconnect button on the Slider screen, two toggles were to be included, the Notification toggle and the Auto Mode toggle. The Note toggle was to utilize another plugin, the Notification Listener plugin by Coconauts, that was supposed to create an event listener upon app start up that would always listen out for notifications sent to the phone's StatusBar, but only use the ones that were caught while the toggle was on its "true" position. When a notification was caught in this mode, the app would send another signal to the Arduino, whose sketch would interpret the signal and cause a ten second blinking to happen with the 12th LED on the ring light. This was to serve as a "notifier" to the user when the user isn't paying attention to their phone but wanted to be updated about their phone notifications. This ultimately failed because of the difficulty in getting the Notification Listener to incorporate, without destroying the rest of the code's functions. The Auto Mode, when toggled, would have just made the sliders disappear, and the phone would send a single

signal to the Arduino that would initiate a loop. This loop would cycle through all of the LED colors while the toggle was selected, and would stop once the toggle was unselected and the sliders returned.

Appendices

Documentation for the systems sketches and app components

Arduino Sketch:

```

#include <SoftwareSerial.h>

int bluetoothTx = 10; // TX-O pin of bluetooth module
int bluetoothRx = 11; // RX-I pin of bluetooth module

SoftwareSerial bluetooth(bluetoothTx, bluetoothRx);

//
//
///START OF NEOPIXEL LIBRARY INFO
#include <Adafruit_NeoPixel.h>
#ifdef __AVR__
  #include <avr/power.h>
#endif

// Which pin on the Arduino is connected to the NeoPixels?
#define PIN      6

// How many NeoPixels are attached to the Arduino?
#define NUMPIXELS 12

Adafruit_NeoPixel pixels = Adafruit_NeoPixel(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);

int delayval = 200; //short initial delay
int bdelay = 1000;
int brepeats = 10;

///END OF NEOPIXEL LIBRARY INFO
//
//
void setup(){

  pixels.begin(); // This initializes the NeoPixel library.
  Serial.begin(9600);
  bluetooth.begin(9600);
  bluetooth.println("Hello, world?");
}

void loop(){

  int red = 0;
  int green = 0;
  int blue = 0;

```

Arduino Sketch(cont):

```

//for custom mode
if (bluetooth.find("c")) {
  red = bluetooth.parseInt(); // parses numeric characters before the comma

  green = bluetooth.parseInt();// parses numeric characters after the comma

  blue = bluetooth.parseInt(); // parses numeric characters after the comma

  Serial.print("Setting color to: " ); // print to console for debugging

  Serial.print(red);Serial.print(", ");

  Serial.print(green);Serial.print(", ");

  Serial.println(blue);

//fills the ring with color
for(int i = 0; i < NUMPIXELS-1; i++){
  pixels.setPixelColor(i, pixels.Color(red, green, blue));
  pixels.show();
  delay(delayval);
}
  Serial.print(red);Serial.print(", ");
  Serial.print(green);Serial.print(", ");
  Serial.print(blue);
}

//this is for automode code
if (bluetooth.find("a")) {
  //the next four lines resets the important rgb variables, as well as sets the "switch" for automode
  int automode = 1;
  red = 0;
  blue = 0;
  green = 0;

  while(automode != 0){
    if(bluetooth.find("s")){
      automode = 0;
    }

    if(red == 0 && green == 0 && blue == 0)
    {

```

Arduino Sketch(cont):

```

red+25;
  for(int i = 0; i < NUMPIXELS; i++){
    pixels.setPixelColor(i, pixels.Color(red, green, blue));
    pixels.show();
    delay(delayval);
  }
}
else if(red > 0 && red < 250 && green == 0 && blue == 0)
{
  red+25;
  for(int i = 0; i < NUMPIXELS; i++){
    pixels.setPixelColor(i, pixels.Color(red, green, blue));
    pixels.show();
    delay(delayval);
  }
}
else if(red == 250 && blue == 0)
{
  while(red > 0){
    red-25;
    green+25;
    for(int i = 0; i < NUMPIXELS; i++){
      pixels.setPixelColor(i, pixels.Color(red, green, blue));
      pixels.show();
      delay(delayval);
    }
  }
}
else if(red == 0 && green <= 250 && blue == 0){
  while(green > 0){
    green-25;
    blue+25;
    for(int i = 0; i < NUMPIXELS; i++){
      pixels.setPixelColor(i, pixels.Color(red, green, blue));
      pixels.show();
      delay(delayval);
    }
  }
}
else if(red == 0 && green == 0 && blue == 250)
{
  while(blue > 0){
    blue-25;
    red+25;
    for(int i = 0; i < NUMPIXELS; i++){

```


Arduino Sketch(cont):

```

        pixels.setPixelColor(i, pixels.Color(red, green, blue));
        pixels.show();
        delay(delayval);
    }
}
}
}
}
//notifications alert
if (bluetooth.find("n"))
{
    while(brepeats)
    {
        if(brepeats % 2 != 0)
        {
            pixels.setPixelColor(NUMPIXELS, pixels.Color(200, 200, 200));
            pixels.show();
            delay(bdelay);
        }
        if(brepeats % 2 == 0)
        {
            pixels.setPixelColor(NUMPIXELS, pixels.Color(0, 0, 0));
            pixels.show();
            delay(bdelay);
        }
        repeats--;
    }
    repeats = 10;
}
}

```

LovellLight Ionic Files

Index.html:

```

<html ng-app="starter">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="initial-scale=1, maximum-scale=1, user-scalable=no,
width=device-width">
    <title></title>
    <link href="lib/ionic/css/ionic.css" rel="stylesheet">
      <link rel="stylesheet" type="text/css" href="css/topcoat-mobile-dark.min.css">
    <link href="css/style.css" rel="stylesheet">

    <!-- IF using Sass (run gulp sass first), then uncomment below and remove the CSS includes above
    <link href="css/ionic.app.css" rel="stylesheet">
    -->

    <!-- ionic/angularjs js -->
    <script src="lib/ionic/js/ionic.bundle.js"></script>

      <!--ng-cordova Script -->
      <script src="js/ng-cordova.min.js"></script>
    <!-- cordova script (this will be a 404 during development) -->
    <script src="cordova.js"></script>

    <!-- your app's js -->
    <script src="js/app.js"></script>
      <script src="js/jquery-1.9.1.min.js"></script>
  </head>
  <body ng-controller="MainCtrl">
    <ion-pane>
      <ion-header-bar class="bar-dark">
        <h1 class="title">LovellLight Companion</h1>
      </ion-header-bar>
      <ion-content>
        <br>
        <br>
        <br>
        <div id="connectionScreen">
          <div>
            <div id="bluetoothconnection">Switch on Bluetooth:</div>
            <!-- <br> -->
            <div style="margin: 30px">

```

Index.html(cont):

```

        <button class="button button-large button-dark"
id="blueswitch">Blueswitch</button>
    </div>
    </br>
    Paired Bluetooth Devices:
    </br>
    </div>
    <ul id="deviceList" class="topcoat-list">
    </ul>
        <div style="margin: 10px">
            <button class="topcoat-button" id="refreshButton">Refresh</button>
        </div>
            <div id="messageDiv"></div>
        </br>
        <div>
            Unpaired Bluetooth Devices:
            </br>
            </div>
                <ul id="udeviceList" class="topcoat-list">
                </ul>
            <div style="margin: 10px">
                <button class="topcoat-button" id="urefreshButton">Refresh</button>
            </div>
                <div id="umessageDiv"></div>
        </div>
    <div id="sliderScreen">
        </br>
        </br>
        <div class="rangeDiv">Red
            <input type="range" min="0" max="255" value="0" name="red" id="red" class="topcoat-
range"/>
        </div>
            </br>
            </br>
            <div class="rangeDiv">Green
                <input type="range" min="0" max="255" value="0" name="green" id="green"
class="topcoat-range"/>
            </div>
                </br>
                </br>
                <div class="rangeDiv">Blue
                    <input type="range" min="0" max="255" value="0" name="blue" id="blue" class="topcoat-
range"/>
                </div>

        <div id="rgbText">

```

```
        &nbsp;
    </div>
    <div>
        <a class="topcoat-button--large" id="settingsend">Send Color</a>
    </div>
</br>
</br>
    <div>
        <a class="topcoat-button--large" id="disconnectButton">Disconnect</a>
    </div>
</div>
<script type="text/javascript" src="cordova.js"></script>
<script type="text/javascript" src="js/underscore-min.js"></script>
<script type="text/javascript" src="js/app.js"></script>
<script type="text/javascript">
    app.start();
</script>
</ion-content>
</ion-pane>
</body>
</html>
```

App.js

```
// Ionic Starter LLc
```

```
// angular.module is a global place for creating, registering and retrieving Angular modules
// 'starter' is the name of this angular module example (also set in a <body> attribute in index.html)
// the 2nd parameter is an array of 'requires'
angular.module('starter', ['ionic'])
```

```
.run(function($ionicPlatform) {
  $ionicPlatform.ready(function() {
    if(window.cordova && window.cordova.plugins.Keyboard) {
      // Hide the accessory bar by default (remove this to show the accessory bar above the keyboard
      // for form inputs)
      cordova.plugins.Keyboard.hideKeyboardAccessoryBar(true);

      // Don't remove this line unless you know what you are doing. It stops the viewport
      // from snLLcing when text inputs are focused. Ionic handles this internally for
      // a much nicer keyboard experience.
      cordova.plugins.Keyboard.disableScroll(true);
    }
    if(window.StatusBar) {
      StatusBar.styleDefault();
    }
  });
});
var app = {
  start: function() {
    this.bindFuncs();
    console.log("Initializing app.");
  },
  bindFuncs: function(){
    console.log("Binding events");
    document.addEventListener('deviceready', this.deviceready, false);
  },
  sliderScreen.hidden = true;
  //the device is ready for use
  deviceready: function() {
    //these three lines connect the buttons to their functions
    console.log("Device ready");

    deviceList.ontouchstart = app.connect;
```

```
udeviceList.ontouchstart = app.connect;
```

App.js(cont):

```
refreshButton.ontouchstart = app.listpaired;
```

```
urefreshButton.ontouchstart = app.scannew;
```

```
disconnectButton.ontouchstart = app.disconnect;
```

```
blueswitch.ontouchstart = app.enable;
```

```
settingsend.ontouchstart = app.onCChange;
```

```
app.enabled();
```

```
},
```

```
enabled: function(){
```

```
    bluetoothSerial.isEnabled(
```

```
        function(){
```

```
            app.listpaired();
```

```
            bluetoothconnection.innerText = "Bluetooth: On";
```

```
            blueswitch.innerText = "BluetoothSwitch";
```

```
            console.log("Bluetooth is enabled.");
```

```
        },
```

```
        function(){
```

```
            console.log("Bluetooth is not enabled");
```

```
            bluetoothconnection.innerText = "Bluetooth: Off";
```

```
            blueswitch.innerText = "BluetoothSwitch";
```

```
        }
```

```
    );
```

```
},
```

```
//searching for available devices
```

```
listpaired: function(event){
```

```
    deviceList.firstChild.innerHTML = "Discovering...";
```

```
    app.setStatus("Looking for Bluetooth Devices.");
```

```
    //if the devices are found, list the devices. else, print out a failed message
```

```
    bluetoothSerial.list(app.ondicelist, app.Failed("Listing Failed"));
```

```
},
```

```
enable: function() {
```

App.js(cont):

```
    bluetoothSerial.enable(
```

```
        function() {
```

```

        alert("Bluetooth has been enabled!");
        bluetoothconnection.innerText = "Bluetooth: On";
        blueswitch.innerText = "BluetoothSwitch";
        app.listpaired();
    },
    function() {
        alert("Bluetooth has been disabled.");
        bluetoothconnection.innerText = "Bluetooth: Off";
        blueswitch.innerText = "hSwitch";
    }
);
},
//connecting to the device
connect: function (e){
    app.setStatus("Connecting");
    app.ustatusset("Connecting");
    var device = e.target.getAttribute('deviceId');
    console.log("Attempting to connect to" + device);
    bluetoothSerial.connect(device, app.onconnect, app.ondisconnect);
},
//disconnection from device
disconnect: function(event) {
    if (event) {
        event.preventDefault();
    }

    app.setStatus("Disconnecting");
    app.ustatusset("Disconnecting");
    bluetoothSerial.disconnect(app.ondisconnect);
},
//printed to app on connection
onconnect: function() {
    connectionScreen.hidden = true;
    sliderScreen.hidden = false;
    app.setStatus("Connected.");
    app.ustatusset("Connected");
    send = 1;
},
//printed on disconnection
ondisconnect: function() {
    connectionScreen.hidden = false;
    sliderScreen.hidden = true;
    send = 0;

```

App.js(cont):

```

app.setStatus("Disconnected.");
app.ustatusset("Disconnected");

```

```

    },
    //handles the color changes done by the sliders
    onCChange: function (evt) {
        var c = app.ColorChange();
        rgbText.innerText = c;

        app.ArduinoConnect(c);
    },
    ColorChange: function(){
        var color = [];
        color.push(red.value);
        color.push(green.value);
        color.push(blue.value);
        return color.join(',');
    },
    ArduinoConnect: function(c){
        bluetoothSerial.write("c" + c + "\n");
    },
    timeoutId: 0,
    setStatus: function(status) {
        if (app.timeoutId) {
            clearTimeout(app.timeoutId);
        }
        messageDiv.innerText = status;
        app.timeoutId = setTimeout(function() { messageDiv.innerText = ""; }, 1000);
    },
    ustatusset: function(status) {
        if (app.timeoutId) {
            clearTimeout(app.timeoutId);
        }
        umessageDiv.innerText = status;
        app.timeoutId = setTimeout(function() { umessageDiv.innerText = ""; }, 1000);
    },
    //creating the list for the devices found
    ondevicelist: function(devices) {
        var listItem, deviceId;

        //remove existing devices
        deviceList.innerHTML = "";
        app.setStatus("");

        //for each device found
        devices.forEach(function(device){

```

App.js(cont):

```

        //creat a list item for it
        listItem = document.createElement ('li');

```



```

        listItem.className = "topcoat-list__item";
        //if the device has a uuid, set it as the id
        if (device.hasOwnProperty("uuid")){
            deviceId = device.uuid;
        } //else, set the address as the id
        else if (device.hasOwnProperty("address")){
            deviceId = device.address;
        } //or print an error message, as the address and the uuid aren't recoverable
        else {
            deviceId = "ERROR" + JSON.stringify(device);
        }
        listItem.setAttribute('deviceId', deviceId);
        listItem.innerHTML = device.name + "<br/><i>" + deviceId + "</i>";
        //put the details for the listItem on the deviceList
        deviceList.appendChild(listItem);
    });

    if (devices.length === 0) {
        app.setStatus("Please Pair a Bluetooth Device.");
    }
    else {
        app.setStatus("Previously Paired: Found " + devices.length + " device" + (devices.length ===
1 ? "." : "s.));
    }
},
scannew: function () {
    //remove existing devices
    udeviceList.innerHTML = "";
    app.ustatusset("");
    bluetoothSerial.discoverUnpaired(function(devices) {
        //remove existing devices
        udeviceList.innerHTML = "Discovering...";
        app.ustatusset("Searching for bluetooth devices...");
        var ulistItem, udeviceId;

        bluetoothSerial.setDeviceDiscoveredListener(function(device) {
            //for each device found
            //creat a list item for it
            ulistItem = document.createElement ('li');

```

App.js(cont):

```

        ulistItem.className = "topcoat-list__item";
        //if the device has a uuid, set it as the id
        if (device.hasOwnProperty("uuid")){
            udeviceId = device.uuid;

```

```

        //else, set the address as the id
        } else if (device.hasOwnProperty("address")){
            udeviceId = device.address;
        //or print an error message, as the address and the uuid aren't recoverable
        } else {
            udeviceId = "ERROR" + JSON.stringify(device);
        }
        ulistItem.setAttribute('deviceId', device.address);
        ulistItem.innerHTML = device.name + "<br/><i>" + udeviceId + "</i>";
        //put the details for the listItem on the deviceList
        udeviceList.appendChild(ulistItem);

    });

    if (devices.length === 0) {
        app.ustatusset("Make sure BT is on, then Pair a Bluetooth Device.");
    }
    else {
        app.ustatusset("Done Searching! Found " + devices.length + " device" +
(devices.length === 1 ? "." : "s.));
    }
});

},
//Listing failed for finding bluetooth devices, or bluetooth device uuid and address wasn't
recoverable
Failed: function(failedlist){
    var fail = function(problem){
        //empty details first
        var details = "";
        if (problem){
            //if there was a problem, fill in the details
            details += ": " +JSON.stringify(problem);
        }
        //then setStatus to the details
        app.setStatus(failedlist + details);
    };
    return fail;
}

};

```

PhoneGap Bluetooth Serial.js

```
/*global cordova*/
module.exports = {

  connect: function (macAddress, success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "connect", [macAddress]);
  },

  // Android only - see http://goo.gl/1mFjZY
  connectInsecure: function (macAddress, success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "connectInsecure", [macAddress]);
  },

  disconnect: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "disconnect", []);
  },

  // list bound devices
  list: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "list", []);
  },

  isEnabled: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "isEnabled", []);
  },

  isConnected: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "isConnected", []);
  },

  // the number of bytes of data available to read is passed to the success function
  available: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "available", []);
  },

  // read all the data in the buffer
  read: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "read", []);
  },

  // reads the data in the buffer up to and including the delimiter
  readUntil: function (delimiter, success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "readUntil", [delimiter]);
  },
}
```

PhoneGap Bluetooth Serial.js(cont):

```

// writes data to the bluetooth serial port
// data can be an ArrayBuffer, string, integer array, or Uint8Array
write: function (data, success, failure) {

    // convert to ArrayBuffer
    if (typeof data === 'string') {
        data = stringToArrayBuffer(data);
    } else if (data instanceof Array) {
        // assuming array of interger
        data = new Uint8Array(data).buffer;
    } else if (data instanceof Uint8Array) {
        data = data.buffer;
    }

    cordova.exec(success, failure, "BluetoothSerial", "write", [data]);
},

// calls the success callback when new data is available
subscribe: function (delimiter, success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "subscribe", [delimiter]);
},

// removes data subscription
unsubscribe: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "unsubscribe", []);
},

// calls the success callback when new data is available with an ArrayBuffer
subscribeRawData: function (success, failure) {

    successWrapper = function(data) {
        // Windows Phone flattens an array of one into a number which
        // breaks the API. Stuff it back into an ArrayBuffer.
        if (typeof data === 'number') {
            var a = new Uint8Array(1);
            a[0] = data;
            data = a.buffer;
        }
        success(data);
    };
    cordova.exec(successWrapper, failure, "BluetoothSerial", "subscribeRaw", []);
},

```

PhoneGap Bluetooth Serial.js(cont):

```

// removes data subscription
unsubscribeRawData: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "unsubscribeRaw", []);
},

// clears the data buffer
clear: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "clear", []);
},

// reads the RSSI of the *connected* peripheral
readRSSI: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "readRSSI", []);
},

showBluetoothSettings: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "showBluetoothSettings", []);
},

enable: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "enable", []);
},

discoverUnpaired: function (success, failure) {
    cordova.exec(success, failure, "BluetoothSerial", "discoverUnpaired", []);
},

setDeviceDiscoveredListener: function (notify) {
    if (typeof notify !== 'function')
        throw 'BluetoothSerial.setDeviceDiscoveredListener: Callback not a function';

    cordova.exec(notify, null, "BluetoothSerial", "setDeviceDiscoveredListener", []);
},

clearDeviceDiscoveredListener: function () {
    cordova.exec(null, null, "BluetoothSerial", "clearDeviceDiscoveredListener", []);
},

setName: function (newName) {
    cordova.exec(null, null, "BluetoothSerial", "setName", [newName]);
},

setDiscoverable: function (discoverableDuration) {
    cordova.exec(null, null, "BluetoothSerial", "setDiscoverable", [discoverableDuration]);
}

```

PhoneGap Bluetooth Serial.js(cont):

```
};  
  
var stringToArrayBuffer = function(str) {  
  var ret = new Uint8Array(str.length);  
  for (var i = 0; i < str.length; i++) {  
    ret[i] = str.charCodeAt(i);  
  }  
  return ret.buffer;  
};
```

LovellLight Lamp Shell Prototype Sketches and Final

